LEARNING GUIDE · MINI PROJECT AI

Knight Trap

Implementasi AI pada Game Knight Trap Menggunakan Algoritma Minimax dengan Alpha-Beta Pruning

Shatrya Christiano Chandra 0706012410064

Bagianmu: Representasi papan, logika aturan, DFS (flood fill), penyatuan evaluasi, playtest

Artificial Intelligence · Information Management & Technology · Universitas Ciputra 2026/2027
Disusun dari proposal (v1.1) dan kode di repo KnightTrap · 3 November 2026

Daftar Isi

1. Peta Belajarmu
2. Gambaran Proyek (Proposal Bab 1)
3. Aturan Game → Kode (Proposal Bab 2.1)
4. Alur Satu Giliran di Kode
5. Konsep AI yang Wajib Dipahami Semua Anggota
6. Representasi Papan dan Setup Proyek
7. Logika Aturan (rules.js)
8. DFS: Depth-First Search dan Flood Fill
9. Menyatukan BFS dan DFS di evaluate()
10. Pengujian Aturan
11. Panduan Playtest (Tabel 2.2.3, baris 4)
12. Hasil Pengujian (Proposal Bab 2.2)
13. Tambahan di Luar Proposal: Makan dan Waktu Giliran
14. Latihan
15. Pertanyaan Presentasi yang Mungkin Muncul
16. Checklist Sebelum Presentasi

1 Peta Belajarmu

Setelah mempelajari guide ini, kamu bisa:

- Menjelaskan representasi papan (array 49 kotak) dan cara mengubah koordinat a1–g7 menjadi indeks.
- Menjelaskan setiap fungsi aturan di `rules.js` dan bagaimana game memastikan langkah sah.
- Menjelaskan DFS/flood fill dan cara mendeteksi papan yang sudah terbelah.
- Menjelaskan bagaimana BFS dan DFS disatukan di `evaluate()`.
- Menjalankan playtest dan mencatat hasilnya dengan benar.

Tugasmu di Timeline (Bab 2.3)

Minggu	Pekerjaan	Kode
1	Setup proyek HTML/CSS/JS, representasi papan	`index.html`, `config.js`, `createState()`
2	Logika aturan: lompatan legal, bakar, jebakan, makan, cek kalah	`rules.js`; `game.js`: `handleCell()`
2–3	DFS (flood fill): cek papan terbelah, luas area	`ai.js`: `dfs()`
3	Penyatuan BFS + DFS ke evaluasi (bersama Clarrence)	`ai.js`: `evaluate()`
3	Playtest ±10 orang, kuesioner, perbaikan bug	Lampiran di guide ini

2 Gambaran Proyek (Proposal Bab 1)

Apa yang kita buat

Knight Trap adalah varian dari permainan papan *Isolation* (1972). Dua knight ada di papan 7 × 7. Pada gilirannya, pemain:

1. **Lompat**: menggerakkan knight seperti kuda catur (huruf L) ke kotak yang belum terbakar atau dijebak.
2. **Bakar**: kotak yang ditinggalkan langsung terbakar dan tidak bisa dipakai lagi.
3. **Jebak**: menghancurkan satu kotak terbuka lain di mana pun.

Pemain yang **tidak punya lompatan legal** pada gilirannya kalah. Di versi kode, mendarat tepat di kotak knight musuh juga langsung menang (**makan**, tambahan di luar proposal; lihat bab "Tambahan di Luar Proposal"). Game ini dibuat untuk **rally games** (NPLC Universitas Ciputra, tema *Human vs AI*), jadi aturannya harus bisa dijelaskan dalam satu menit, satu game selesai dalam beberapa menit, dan lawan AI-nya tidak boleh terlalu mudah.

Masalah yang diselesaikan (Bab 1.2)

Kita harus membuat lawan AI yang (1) memilih langkah yang kuat, (2) cukup cepat untuk browser, dan (3) punya beberapa tingkat kesulitan. Ada dua tantangan utama:

- **Pohon permainan besar.** Satu langkah = pilihan lompatan (maks. 8) × pilihan kotak jebakan (bisa puluhan). Jumlah posisi meledak setiap giliran.
- **Menilai posisi tanpa main sampai habis.** Kita butuh *fungsi evaluasi* yang memberi skor pada posisi di tengah permainan.

Solusinya, sesuai ketentuan tugas yang membatasi kita ke **empat algoritma**: **Minimax** + **Alpha-Beta Pruning** untuk mengambil keputusan, dan **BFS** + **DFS** sebagai dasar fungsi evaluasi.

Peta file kode

File	Isi	Pemilik utama (Timeline)
`js/config.js`	Konstanta: ukuran papan, posisi awal, kedalaman tiap level, lama satu giliran, batas AI	Shatrya (setup), Louis (AI)
`js/rules.js`	Aturan game: lompatan legal, bakar, jebakan, makan, cek kalah. Tidak menyentuh DOM	Shatrya
`js/ai.js`	Minimax + Alpha-Beta, BFS, DFS, fungsi evaluasi, pemain acak & greedy	Louis (search), Clarrence (BFS), Shatrya (DFS)
`js/ai.worker.js`, `js/ai-client.js`	Menjalankan AI di Web Worker supaya tampilan tidak macet	Louis
`js/game.js`	Alur giliran: input pemain, giliran AI, penghitung waktu giliran, game over	Shatrya & Louis
`js/board.js`, `css/style.css`, `index.html`	Tampilan papan, knight, penanda lompatan, layout	Clarrence
`js/main.js`, `js/i18n.js`	Layar menu, pengaturan, Cara Bermain, panel statistik, teks dua bahasa	Clarrence & Louis
`js/net.js`	Multiplayer online (PeerJS)	Tambahan di luar proposal
`tests/run.mjs`	Semua pengujian Bab 2.2, plus uji makan dan waktu giliran	Louis, Clarrence (pengujian)

> **Repo dengan komentar per baris.** Folder `KnightTrap-Annotated` berisi kode yang sama persis, tetapi setiap baris diberi komentar bahasa Indonesia. Pakai repo itu untuk belajar; potongan kode di guide ini diambil dari repo asli `KnightTrap` supaya nomor barisnya cocok dengan kode yang dijalankan.

3 Aturan Game → Kode (Proposal Bab 2.1)

Setiap aturan di proposal punya tempat yang jelas di `js/rules.js`. File ini adalah **satu-satunya sumber aturan**: tampilan, AI, dan mode online semuanya memakai fungsi yang sama.

Aturan proposal	Kode
Papan 7 × 7, array 49 kotak (kosong / terbakar / jebakan) + posisi 2 knight	`createState()`, konstanta `EMPTY`, `BURNED`, `TRAP`
(a) Knight bergerak huruf L	`JUMPS` dan `NEIGHBORS` (dihitung sekali di awal)
(b) Kotak yang ditinggalkan terbakar	`applyJump()`
(c) Hancurkan satu kotak terbuka	`applyTrap()`; validasi di `validateMove()`
(d) Tidak boleh mendarat di kotak terbakar atau jebakan. Proposal juga melarang kotak lawan; di kode, mendarat di sana = makan (tambahan)	`legalJumps()` (cek `cells[n] === EMPTY`); `isOpen()` untuk kotak jebakan
(e) Tidak punya lompatan legal = kalah	`winner()` memakai `canMove()`
Tambahan: dimakan = kalah, langkah makan tanpa jebakan	`winner()` (`pos[0] === pos[1]`), `validateMove()`

Representasi papan

Kotak diberi nomor 0 sampai 48: `index = baris × 7 + kolom`, baris 0 adalah baris paling atas (rank 7). Jadi **a1 = 42** (kiri bawah) dan **g7 = 6** (kanan atas).

js/config.js : baris 3–8

```
3  export const N = 7;
4  export const SIZE = N * N;
5
6  // The proposal does not fix start squares. P1 starts at a1 (bottom-left),
7  // P2 at g7 (top-right). Index = row * 7 + col, row 0 is the top rank (7).
8  export const START = Object.freeze({ p1: 42, p2: 6, first: 0 });
```

js/rules.js : baris 28–35

```
28 export function createState(start = START) {
29   return {
30     cells: new Int8Array(SIZE),
31     pos: [start.p1, start.p2],
32     turn: start.first ?? 0,
33     moves: 0,
34   };
35 }
```

`cells` memakai `Int8Array` (array angka 8-bit) karena kecil dan cepat disalin. `pos[0]` adalah posisi knight P1, `pos[1]` knight P2, `turn` adalah siapa yang jalan (0 atau 1).

Tetangga knight dihitung sekali

js/rules.js : baris 10–26

```
const JUMPS = [
  [-2, -1], [-2, 1], [-1, -2], [-1, 2],
  [1, -2], [1, 2], [2, -1], [2, 1],
];

// Knight neighbor lists, computed once.
export const NEIGHBORS = Array.from({ length: SIZE }, (_, i) => {
  const r = Math.floor(i / N);
  const c = i % N;
  const out = [];
  for (const [dr, dc] of JUMPS) {
    const rr = r + dr;
    const cc = c + dc;
    if (rr >= 0 && rr < N && cc >= 0 && cc < N) out.push(rr * N + cc);
  }
  return out;
});
```

Untuk setiap kotak, kita coba 8 arah L dan simpan hanya yang masih di dalam papan. Hasilnya tabel 49 daftar. AI memanggil daftar ini jutaan kali, jadi menghitungnya sekali di awal menghemat banyak waktu.

Satu giliran = lompat, bakar, jebak

js/rules.js : baris 54–56

```
export function isOpen(s, sq) {
  return s.cells[sq] === EMPTY && sq !== s.pos[0] && sq !== s.pos[1];
}
```

js/rules.js : baris 60–64

```
export function legalJumps(s, side = s.turn) {
  const out = [];
  for (const n of NEIGHBORS[s.pos[side]]) if (s.cells[n] === EMPTY) out.push(n);
  return out;
}
```

js/rules.js : baris 77–83

```
export function applyJump(s, to) {
  const side = s.turn;
  const from = s.pos[side];
  s.cells[from] = BURNED;
  s.pos[side] = to;
  return from;
}
```

js/rules.js : baris 86–88

```
export function applyTrap(s, sq) {
  if (sq >= 0) s.cells[sq] = TRAP;
}
```

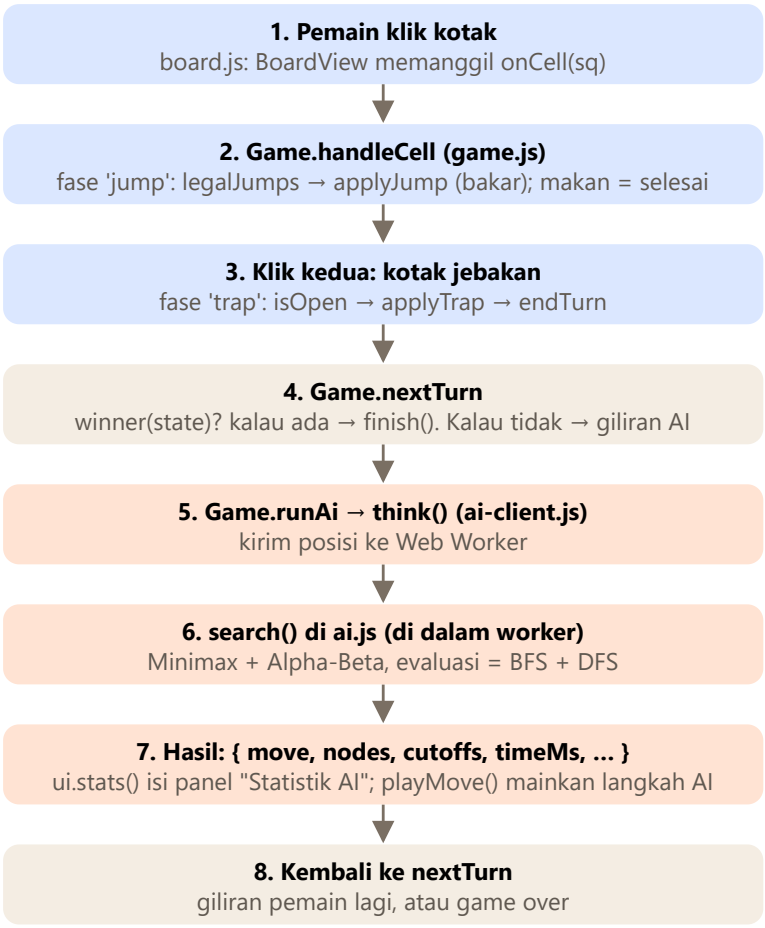
js/rules.js : baris 116–119

```
116  export function winner(s) {
117    if (s.pos[0] === s.pos[1]) return s.turn ^ 1;
118    return canMove(s, s.turn) ? null : s.turn ^ 1;
119  }
```

Kenapa `winner()` mengembalikan `s.turn ^ 1`? `^` adalah XOR. `0 ^ 1 = 1` dan `1 ^ 1 = 0`, jadi ini cara singkat menulis "pemain lainnya". Kalau yang sedang jalan tidak bisa bergerak, atau baru saja dimakan (kedua knight di kotak yang sama), lawannya menang.

4 Alur Satu Giliran di Kode

Bagian ini menghubungkan semua file. Setiap anggota harus bisa menceritakan alur ini saat presentasi.



Gambar: alur satu ronde (pemain lalu AI). Biru = input pemain, krem = alur game, oranye = AI.

js/game.js : baris 127–146

```
127   nextTurn() {
128     if (this.disposed) return;
129     const w = winner(this.state);
130     if (w !== null) return this.finish(w);
131     const p = this.current;
132     if (p.kind === 'human') {
133       this.phase = 'jump';
134       this.startClock();
135       this.refresh();
136     } else if (p.kind === 'ai') {
137       this.phase = 'ai';
138       this.stopClock();
139       this.refresh();
140       this.runAi();
141     } else {
142       this.phase = 'remote';
143       this.startClock();
144       this.refresh();
145     }
146   }
```

js/game.js : baris 209–233

```
  async runAi() {
    this.ui.thinking(true);
    const t0 = performance.now();
    let res = null;
    try {
      res = await think(this.state, { depth: this.ai.depth, prune: this.ai.prune });
    } catch (err) {
      console.error('AI failed', err);
    }
    if (this.disposed) return;
    const elapsed = performance.now() - t0;
    if (elapsed < AICFG.minUiDelayMs) await sleep(AICFG.minUiDelayMs - elapsed);
    if (this.disposed) return;
    this.ui.thinking(false);
    let move = res?.move;
    if (!move || !validateMove(this.state, move)) {
      console.warn('AI returned no valid move; falling back to a random legal move');
      move = randomMove(this.state);
    } else {
      this.ui.stats(res);
    }
    await this.playMove(move);
    if (this.disposed) return;
    this.nextTurn();
  }
```

Perhatikan dua pengaman di `runAi`: (1) kalau AI terlalu cepat (Easy), tampilan menunggu minimal 450 ms supaya tidak terasa "langsung"; (2) kalau AI gagal atau mengembalikan langkah tidak sah, game memakai langkah acak yang legal, jadi game tidak pernah macet.

js/ai-client.js : baris 44–53

```
export function think(state, { depth, prune }) {
  const args = { state: serialize(state), depth, prune };
  const w = getWorker();
  if (!w) return runInline(args);
  return new Promise((resolve, reject) => {
    const id = ++seq;
    pending.set(id, { resolve, reject, args });
    w.postMessage({ id, ...args });
  });
}
```

js/ai.worker.js : baris 5–13

```
self.onmessage = (e) => {
  const { id, state, depth, prune } = e.data;
  try {
    const result = search(deserialize(state), { depth, prune });
    self.postMessage({ id, result });
  } catch (err) {
    self.postMessage({ id, error: String(err && err.message ? err.message : err) });
  }
};
```

Kenapa Web Worker? JavaScript di browser berjalan di satu thread. Kalau Expert berpikir 1–2 detik di thread utama, tombol dan penghitung waktu akan membeku. Worker menjalankan `search()` di thread terpisah, jadi papan tetap hidup saat AI berpikir.

5 Konsep AI yang Wajib Dipahami Semua Anggota

Dosen bisa bertanya ke siapa saja. Bagian ini adalah dasar yang harus dikuasai semua anggota, walaupun detailnya milik anggota lain.

Agen dan model P.E.A.S (Tabel 2.1.1)

Komponen	Di Knight Trap	Di kode
Performance	Win rate AI, waktu berpikir ≤ 4 detik, jumlah posisi diperiksa sedikit, tantangan pas untuk siswa SMA	`tests/run.mjs`, panel statistik
Environment	Papan 7 × 7, dua agen bergiliran. Deterministik, fully observable, diskret, berurutan, zero-sum	`createState()`
Actuators	Lompat ke kotak legal (maks. 8 arah) + hancurkan satu kotak	`applyMove()`, `{ jump, trap }`
Sensors	Posisi kedua knight, status 49 kotak, giliran	`state.cells`, `state.pos`, `state.turn`

Kenapa sifat lingkungan penting? Minimax hanya cocok untuk game *deterministik* (tidak ada dadu), *fully observable* (tidak ada kartu tertutup), dan *zero-sum* (untung saya = rugi lawan). Knight Trap memenuhi ketiganya.

Adversarial search dan game tree

Game tree: akar = posisi sekarang, cabang = langkah, simpul = posisi hasil langkah. **Kedalaman** (depth) = berapa giliran ke depan yang dilihat AI. **Branching factor** = rata-rata jumlah langkah per posisi. Di Knight Trap: sampai 8 lompatan × sampai 12 kandidat jebakan = sampai **96 langkah** per posisi. Dengan depth 4: $96^4 \approx 85$ juta posisi. Itulah alasan kita butuh Alpha-Beta dan pembatasan kandidat.

Minimax dalam satu paragraf

AI adalah **MAX** (ingin skor setinggi mungkin), manusia adalah **MIN** (dianggap selalu memilih langkah terburuk untuk AI). AI menelusuri pohon sampai kedalaman tertentu, menilai posisi di ujung dengan fungsi evaluasi, lalu menaikkan nilai ke atas: di simpul MAX ambil nilai terbesar, di simpul MIN ambil nilai terkecil. Langkah di akar dengan nilai terbesar dipilih.

Alpha-Beta dalam satu paragraf

Alpha = nilai terbaik yang sudah pasti didapat MAX. **Beta** = nilai terbaik yang sudah pasti didapat MIN. Kalau di suatu simpul `beta ≤ alpha`, sisa langkah di simpul itu tidak mungkin mengubah keputusan, jadi dilewati (**cutoff**). Hasilnya **sama persis** dengan Minimax biasa, tapi posisi yang diperiksa jauh lebih sedikit. Ini dibuktikan di pengujian.

BFS dan DFS dalam satu paragraf

BFS (antrian, menyebar per "lapisan") menghitung jumlah lompatan minimum dari knight ke setiap kotak. Kotak yang lebih cepat dicapai knight A adalah **wilayah** A. **DFS** (tumpukan, masuk sedalam mungkin dulu)

melakukan **flood fill**: menandai semua kotak yang bisa dicapai knight. Kalau area kedua knight tidak bersentuhan, papan sudah **terbelah** dan pemenangnya hampir pasti yang areanya lebih luas.

js/ai.js : baris 80–115

```
80  export function evaluate(s, me, ply = 0) {
81    const { cells, pos, turn } = s;
82    const sign = turn === me ? 1 : -1;
83    // The side to move was just eaten, or has no jump: it has lost.
84    if (pos[0] === pos[1] || !hasJump(s, turn)) return -sign * (WIN - ply);
85    // The side to move can eat right now, so it wins on the next ply.
86    if (NEIGHBORS[pos[turn]].includes(pos[turn ^ 1])) return sign * (WIN - ply - 1);
87    const areaA = dfs(cells, pos, 0, markA);
88    const areaB = dfs(cells, pos, 1, markB);
89    let split = true;
90    for (let i = 0; i < SIZE; i++) {
91      if (markA[i] === 1 && markB[i] === 1) {
92        split = false;
93        break;
94      }
95    }
96    let score;
97    if (split) {
98      score = (areaA - areaB) * SPLIT_WEIGHT;
99    } else {
100     bfs(cells, pos, 0, distA);
101     bfs(cells, pos, 1, distB);
102     let terr = 0;
103     for (let i = 0; i < SIZE; i++) {
104       if (!open(cells, pos, i)) continue;
105       const a = distA[i];
106       const b = distB[i];
107       if (a < 0 && b < 0) continue;
108       if (b < 0 || (a >= 0 && a < b)) terr++;
109       else if (a < 0 || b < a) terr--;
110       else terr += turn === 0 ? 1 : -1; // tie goes to the side to move
111     }
112     score = terr + 0.5 * (mobility(cells, pos, 0) - mobility(cells, pos, 1));
113   }
114   return me === 0 ? score : -score;
115 }
```

Inti fungsi evaluasi (skor dari sudut pandang P1, lalu dibalik tandanya untuk P2):

- Cek dulu posisi yang sudah pasti: yang jalan sudah dimakan atau tidak bisa lompat = kalah (`WIN − ply`); yang jalan bisa langsung makan = menang di ply berikutnya (`WIN − ply − 1`).
- Kalau papan sudah terbelah (DFS): `skor = (areaP1 − areaP2) × 100` .
- Kalau belum: `skor = selisih wilayah (BFS) + 0,5 × selisih mobilitas` . Mobilitas = jumlah lompatan legal saat ini.

Angka 100 (`SPLIT_WEIGHT`) dipilih karena skor wilayah paling besar sekitar 53, jadi papan yang sudah terbelah selalu dinilai lebih "pasti" daripada posisi yang masih terbuka.

6 Representasi Papan dan Setup Proyek

Struktur proyek tanpa framework

Game ini HTML, CSS, dan JavaScript murni (ES modules), tanpa framework dan tanpa build step. `index.html` memuat satu script: `<script type="module" src="js/main.js">`; sisanya dimuat lewat `import`. Karena AI berjalan di *module Web Worker*, halaman harus dibuka lewat server (misalnya `npx serve .`), tidak bisa dengan klik ganda file.

Dari koordinat ke indeks

Kotak	Baris (r)	Kolom (c)	Indeks = r × 7 + c
g7 (kanan atas)	0	6	6
d4 (tengah)	3	3	24
a1 (kiri bawah)	6	0	42

Kebalikannya: `r = Math.floor(i / 7)`, `c = i % 7`. Nama kotak dibuat oleh `squareName()`:

js/rules.js : baris 121–123

```
121  export function squareName(sq) {
122    return 'abcdefg'[sq % N] + (N - Math.floor(sq / N));
123  }
```

js/rules.js : baris 42–44

```
42  export function serialize(s) {
43    return { cells: Array.from(s.cells), pos: [s.pos[0], s.pos[1]], turn: s.turn, moves: s.moves };
44  }
```

js/rules.js : baris 46–51

```
46  export function deserialize(o) {
47    if (!o || !Array.isArray(o.cells) || o.cells.length !== SIZE || !Array.isArray(o.pos)) {
48      throw new Error('bad state');
49    }
50    return { cells: Int8Array.from(o.cells), pos: [o.pos[0] | 0, o.pos[1] | 0], turn: o.turn & 1, moves:
o.moves | 0 };
51  }
```

`Int8Array` tidak bisa dikirim langsung lewat jaringan atau ke worker sebagai JSON biasa, jadi `serialize` mengubahnya ke array biasa. `deserialize` juga **memeriksa** data (panjang harus 49, `pos` harus array) dan melempar error kalau rusak. Ini penting untuk mode online, karena data dari pemain lain tidak boleh dipercaya begitu saja. `| 0` dan `& 1` memaksa nilai menjadi bilangan bulat dan 0/1.

7 Logika Aturan (rules.js)

js/rules.js : baris 66–69

```
66  export function canMove(s, side = s.turn) {
67    for (const n of NEIGHBORS[s.pos[side]]) if (s.cells[n] === EMPTY) return true;
68    return false;
69  }
```

js/rules.js : baris 71–74

```
71  export function hasOpenSquare(s) {
72    for (let i = 0; i < SIZE; i++) if (isOpen(s, i)) return true;
73    return false;
74  }
```

js/rules.js : baris 90–93

```
90  export function endTurn(s) {
91    s.turn ^= 1;
92    s.moves++;
93  }
```

`canMove` sama seperti `legalJumps(...).length > 0`, tetapi berhenti di lompatan legal pertama, jadi lebih cepat. Kotak musuh ikut dihitung, karena melompat ke sana berarti makan; jadi knight yang satu-satunya lompatannya adalah makan tidak dianggap terkurung. `hasOpenSquare` dipakai untuk kasus langka: setelah lompat, tidak ada kotak terbuka yang bisa dijebak. Saat itu jebakan dilewati dan dicatat sebagai `trap = -1`.

Validasi langkah lengkap

js/rules.js : baris 96–105

```
96  export function validateMove(s, move) {
97    if (!move || !Number.isInteger(move.jump) || !Number.isInteger(move.trap)) return false;
98    if (winner(s) !== null) return false;
99    if (!legalJumps(s).includes(move.jump)) return false;
100   if (move.jump === s.pos[s.turn ^ 1]) return move.trap === -1; // eating ends the game, so no trap
101   const t = cloneState(s);
102   applyJump(t, move.jump);
103   if (!hasOpenSquare(t)) return move.trap === -1;
104   return move.trap >= 0 && move.trap < SIZE && isOpen(t, move.trap);
105 }
```

1. `jump` dan `trap` harus bilangan bulat (menolak data aneh dari jaringan).
2. Game belum selesai.
3. Lompatan termasuk `legalJumps`.
4. Kalau lompatan mendarat di kotak musuh (makan), jebakan harus −1 karena game langsung selesai.
5. Lompatan diterapkan ke **salinan** (`cloneState`), supaya state asli tidak berubah.
6. Kalau tidak ada kotak terbuka, jebakan harus −1. Kalau ada, jebakan harus kotak terbuka (bukan kotak knight mana pun, bukan kotak yang baru terbakar).

Fungsi ini dipakai di dua tempat: host online memeriksa langkah tamu, dan game memeriksa langkah AI sebelum dimainkan. Tes juga memakainya untuk memastikan setiap langkah AI sah.

js/rules.js : baris 107–112

```
export function applyMove(s, move) {
  const from = applyJump(s, move.jump);
  applyTrap(s, move.trap);
  endTurn(s);
  return from;
}
```

Input pemain: dua klik per giliran

js/game.js : baris 256–275

```
  handleCell(sq) {
    if (this.disposed || this.current?.kind !== 'human') return;
    const s = this.state;
    if (this.phase === 'jump') {
      if (!legalJumps(s).includes(sq)) return;
      const from = applyJump(s, sq);
      this.pendingFrom = from;
      this.pendingJump = sq;
      this.lastLand = sq;
      this.lastTrap = null;
      if (sq === s.pos[s.turn ^ 1] || !hasOpenSquare(s)) return this.completeHumanMove(-1);
      this.phase = 'trap';
      this.refresh();
    } else if (this.phase === 'trap') {
      if (!isOpen(s, sq)) return;
      applyTrap(s, sq);
      this.lastTrap = sq;
      this.completeHumanMove(sq);
    }
  }
```

Fase `jump`: klik di kotak yang bukan lompatan legal diabaikan. Kalau legal, `applyJump` dijalankan dan kotak asal disimpan di `pendingFrom` (untuk tombol batal). Kalau lompatan itu memakan musuh, atau tidak ada kotak tersisa untuk dijebak, giliran langsung selesai dengan jebakan −1. **Fase `trap`:** klik harus di kotak terbuka; lalu `applyTrap` dan `completeHumanMove` mengakhiri giliran. Saat waktu giliran habis, `timeUp()` juga memanggil `handleCell()`, jadi aturan yang sama berlaku.

js/game.js : baris 277–288

```
  undoJump() {
    if (this.phase !== 'trap' || this.pendingFrom == null) return;
    const s = this.state;
    const side = s.turn;
    s.pos[side] = this.pendingFrom;
    s.cells[this.pendingFrom] = EMPTY;
    this.lastLand = null;
    this.pendingFrom = null;
    this.pendingJump = null;
    this.phase = 'jump';
    this.refresh();
  }
```

Batal lompat aman karena giliran baru dihitung setelah jebakan dipasang. Knight dikembalikan, dan kotak asal dibuat `EMPTY` lagi (tidak jadi terbakar). Pemain bisa membatalkan lewat tombol Batal lompat, klik knight sendiri, atau tombol Esc. Waktu giliran tidak diulang.

8 DFS: Depth-First Search dan Flood Fill

Ide

DFS menelusuri satu jalur **sedalam mungkin** sebelum mundur. Struktur datanya **tumpukan** (stack, LIFO): yang terakhir masuk keluar pertama. Di proyek ini DFS dipakai untuk **flood fill**: menandai semua kotak yang bisa dicapai knight lewat rangkaian lompatan, seperti tumpahan cat yang menyebar ke semua kotak yang terhubung.

Untuk flood fill, urutan kunjungan tidak penting (yang penting *semua* kotak terhubung ditemukan), jadi DFS cocok dan sedikit lebih sederhana daripada BFS: tidak perlu menyimpan jarak.

`js/ai.js : baris 58–76`

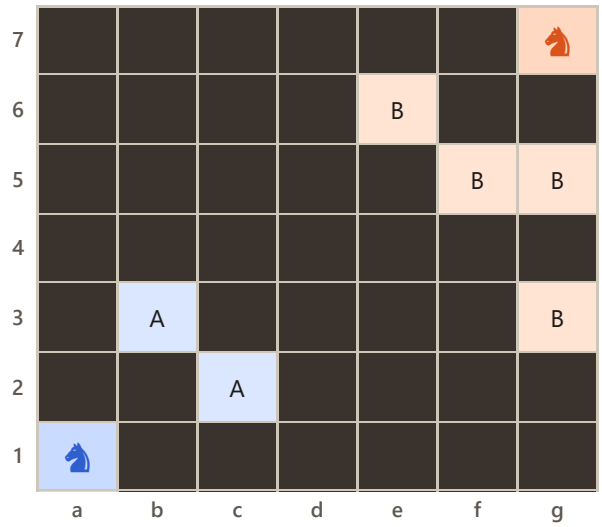
```
58  export function dfs(cells, pos, side, mark) {
59    mark.fill(0);
60    const start = pos[side];
61    mark[start] = 2;
62    let top = 0;
63    let size = 0;
64    stack[top++] = start;
65    while (top > 0) {
66      const cur = stack[--top];
67      for (const n of NEIGHBORS[cur]) {
68        if (mark[n] === 0 && open(cells, pos, n)) {
69          mark[n] = 1;
70          size++;
71          stack[top++] = n;
72        }
73      }
74    }
75    return size;
76  }
```

Baris	Arti
`mark.fill(0)`	Semua kotak belum ditandai.
`mark[start] = 2`	Kotak awal diberi tanda khusus 2 supaya tidak ikut dihitung sebagai area dan tidak dikunjungi lagi.
`stack[top++] = start`	Push kotak awal ke tumpukan.
`stack[--top]`	Pop: ambil kotak paling atas.
`mark[n] === 0 && open(...)`	Tetangga belum ditandai dan masih terbuka → tandai 1, tambah luas, push.
`return size`	Luas area = jumlah kotak yang bisa dicapai (kotak awal tidak dihitung).

> **Kenapa stack berukuran `SIZE * 8`?** Ukuran aman: setiap kotak hanya di-push sekali (karena langsung ditandai), jadi 49 sebenarnya cukup, tetapi buffer lebih besar tidak berbahaya dan dibuat sekali saja.

Mendeteksi papan terbelah

DFS dijalankan dari kedua knight, dengan tanda di `markA` dan `markB`. Kalau tidak ada satu pun kotak yang ditandai 1 di keduanya, kedua knight hidup di area terpisah dan tidak bisa saling mengganggu lagi. Pemenangnya ditentukan oleh siapa yang areanya lebih luas (dia bisa melompat lebih lama).



Contoh papan terbelah (dipakai di tests/run.mjs). Area P1 = 2 kotak, area P2 = 4 kotak, tidak ada kotak yang sama → split. Skor P2 = (4 − 2) × 100 = 200.

js/ai.js : baris 87–98

```
87    const areaA = dfs(cells, pos, 0, markA);
88    const areaB = dfs(cells, pos, 1, markB);
89    let split = true;
90    for (let i = 0; i < SIZE; i++) {
91      if (markA[i] === 1 && markB[i] === 1) {
92        split = false;
93        break;
94      }
95    }
96    let score;
97    if (split) {
98      score = (areaA - areaB) * SPLIT_WEIGHT;
```

9 Menyatukan BFS dan DFS di evaluate()

js/ai.js : baris 80–115

```
80  export function evaluate(s, me, ply = 0) {
81    const { cells, pos, turn } = s;
82    const sign = turn === me ? 1 : -1;
83    // The side to move was just eaten, or has no jump: it has lost.
84    if (pos[0] === pos[1] || !hasJump(s, turn)) return -sign * (WIN - ply);
85    // The side to move can eat right now, so it wins on the next ply.
86    if (NEIGHBORS[pos[turn]].includes(pos[turn ^ 1])) return sign * (WIN - ply - 1);
87    const areaA = dfs(cells, pos, 0, markA);
88    const areaB = dfs(cells, pos, 1, markB);
89    let split = true;
90    for (let i = 0; i < SIZE; i++) {
91      if (markA[i] === 1 && markB[i] === 1) {
92        split = false;
93        break;
94      }
95    }
96    let score;
97    if (split) {
98      score = (areaA - areaB) * SPLIT_WEIGHT;
99    } else {
100     bfs(cells, pos, 0, distA);
101     bfs(cells, pos, 1, distB);
102     let terr = 0;
103     for (let i = 0; i < SIZE; i++) {
104       if (!open(cells, pos, i)) continue;
105       const a = distA[i];
106       const b = distB[i];
107       if (a < 0 && b < 0) continue;
108       if (b < 0 || (a >= 0 && a < b)) terr++;
109       else if (a < 0 || b < a) terr--;
110       else terr += turn === 0 ? 1 : -1; // tie goes to the side to move
111     }
112     score = terr + 0.5 * (mobility(cells, pos, 0) - mobility(cells, pos, 1));
113   }
114   return me === 0 ? score : -score;
115 }
```

1. **Posisi yang sudah pasti dicek dulu:** yang jalan sudah dimakan atau tidak bisa lompat = kalah; yang jalan bisa langsung makan = menang di ply berikutnya. Skornya memakai `WIN - ply`, sama seperti pencarian.
2. **DFS** untuk kedua knight, hitung luas area.
3. **Cek split:** apakah ada kotak yang dicapai keduanya?
4. **Kalau terbelah:** skor = (areaA − areaB) × `SPLIT_WEIGHT` (100).
5. **Kalau belum:** BFS dari kedua knight, hitung wilayah (bagian Clarrence), tambah 0,5 × selisih mobilitas.
6. **Sudut pandang:** skor dihitung untuk P1, lalu tandanya dibalik kalau AI adalah P2.

> **Kenapa bobot 100?** Skor wilayah paling besar sekitar 49 + 4 (mobilitas) ≈ 53. Dengan × 100, selisih area 1 kotak pada papan terbelah (100) selalu lebih besar daripada keunggulan wilayah apa pun. Artinya AI lebih suka posisi yang *sudah pasti* menang daripada posisi terbuka yang hanya kelihatan bagus. Tes memeriksa ini: `SPLIT_WEIGHT > 49 + 4`.

Urutan DFS → BFS juga menghemat waktu: kalau papan sudah terbelah, dua BFS tidak perlu dijalankan.

10 Pengujian Aturan

tests/run.mjs : baris 38–83

```
38  // ---------------------------------------------------------------------------
39  console.log('\n1. Rules');
40  {
41    const s = createState();
42    check('start: P1 on a1', s.pos[0] === sq('a1'));
43    check('start: P2 on g7', s.pos[1] === sq('g7'));
44    check('start: P1 moves first', s.turn === 0);
45    check('a1 legal jumps = b3, c2', JSON.stringify(legalJumps(s).sort((a, b) => a - b)) ===
JSON.stringify([sq('b3'), sq('c2')].sort((a, b) => a - b)));
46
47    const t = cloneState(s);
48    const from = applyJump(t, sq('b3'));
49    check('jump moves knight', t.pos[0] === sq('b3'));
50    check('origin square burns', from === sq('a1') && t.cells[sq('a1')] === BURNED);
51    applyTrap(t, sq('d4'));
52    check('trap marks square', t.cells[sq('d4')] === TRAP);
53    endTurn(t);
54    check('turn passes', t.turn === 1 && t.moves === 1);
55
56    // Cannot land on burned or trap squares. The enemy's square is a legal jump: landing there eats it.
57    const u = createState({ p1: sq('d4'), p2: sq('e6'), first: 0 });
58    u.cells[sq('b3')] = BURNED;
59    u.cells[sq('c6')] = TRAP;
60    const j = legalJumps(u);
61    check('blocked: burned square', !j.includes(sq('b3')));
62    check('blocked: trap square', !j.includes(sq('c6')));
63    check('opponent square is a legal jump', j.includes(sq('e6')));
64    check('opponent square is not open for a trap', !isOpen(u, sq('e6')));
65    check('open squares still legal', j.includes(sq('f5')) && j.includes(sq('b5')));
66
67    // Trap validation.
68    check('valid move accepted', validateMove(u, { jump: sq('f3'), trap: sq('a1') }));
69    check('trap on burned rejected', !validateMove(u, { jump: sq('f3'), trap: sq('b3') }));
70    check('trap on opponent rejected', !validateMove(u, { jump: sq('f3'), trap: sq('e6') }));
71    check('trap on own new square rejected', !validateMove(u, { jump: sq('f3'), trap: sq('f3') }));
72    check('trap on burned origin rejected', !validateMove(u, { jump: sq('f3'), trap: sq('d4') }));
73    check('illegal jump rejected', !validateMove(u, { jump: sq('d5'), trap: sq('a1') }));
74    check('missing trap rejected while squares are open', !validateMove(u, { jump: sq('f3'), trap: -1
}));
75    check('garbage rejected', !validateMove(u, { jump: 'x', trap: 3 }) && !validateMove(u, null));
76
77    // Loss detection: P1 on a1 with b3 and c2 gone loses on its turn.
78    const v = createState();
79    v.cells[sq('b3')] = TRAP;
80    v.cells[sq('c2')] = BURNED;
81    check('no legal jump -> side to move loses', winner(v) === 1 && !canMove(v));
82    v.turn = 1;
83    check('other side still alive', winner(v) === null);
```

Bagian 1 dari tes memeriksa setiap aturan: posisi awal, lompatan legal dari a1 (hanya b3 dan c2), kotak asal terbakar, jebakan tercatat, giliran berpindah, tidak bisa mendarat di kotak terbakar/jebakan, kotak musuh boleh dituju tetapi tidak boleh dijebak, validasi jebakan, deteksi kalah, aturan makan (lihat bab "Tambahan di Luar Proposal"), kasus tidak ada kotak tersisa, serialisasi, dan contoh papan terbelah untuk DFS. Potongan di atas hanya sampai deteksi kalah.

11 Panduan Playtest (Tabel 2.2.3, baris 4)

Target proposal: sekitar 10 siswa SMA atau teman, kuesioner skala 1–5, **rata-rata kejelasan aturan minimal 4** dan tingkat tantangan dinilai sesuai. Tes otomatis tidak bisa menggantikan ini, dan hasilnya hanya boleh dilaporkan kalau benar-benar dilakukan.

Langkah

1. Jelaskan aturan maksimal 1 menit (pakai layar "Cara Bermain"). Catat waktunya.
2. Peserta main 1 game di Easy, lalu 1 game di level pilihannya.
3. Jangan membantu saat bermain; catat di mana peserta bingung.
4. Isi kuesioner, lalu tanyakan satu hal yang paling membingungkan.

Kuesioner (skala 1 = sangat tidak setuju, 5 = sangat setuju)

No	Pernyataan	1	2	3	4	5
1	Aturan game mudah dipahami.					
2	Saya tahu kotak mana yang boleh saya lompati.					
3	Saya mengerti kenapa saya menang atau kalah.					
4	Tingkat kesulitan yang saya pilih terasa menantang tapi adil.					
5	AI terasa "berpikir", bukan bergerak acak.					
6	Saya ingin main lagi.					

Catatan bug

No	Apa yang terjadi	Langkah untuk mengulang	Perangkat / browser	Status
1				
2				
3				

Laporkan: jumlah peserta, rata-rata tiap pernyataan, level yang dipilih, berapa yang menang, dan bug yang ditemukan beserta perbaikannya.

12 Hasil Pengujian (Proposal Bab 2.2)

Semua angka di bawah berasal dari `node tests/run.mjs` (run penuh: 100 game per level per lawan). Hasil akhir run: **900 cek lulus, 0 gagal**. Jalankan sendiri sebelum presentasi supaya kamu tahu prosesnya: `node tests/run.mjs --quick` (±30 detik) atau tanpa `--quick` (±5 menit).

1. Kebenaran Alpha-Beta (Tabel 2.2.3, baris 1)

30 posisi uji dibuat dari permainan acak 4–8 giliran dengan seed `20261028 + k` (bisa diulang siapa pun). Alpha-Beta dan Minimax biasa memakai evaluasi, urutan langkah, dan tanpa batas waktu yang sama; bedanya hanya pemangkasan.

Depth	Skor identik	Langkah identik	Node Minimax	Node Alpha-Beta	Hemat	Cutoff	ms Minimax	ms Alpha-Beta
1	30/30	30/30	981	981	0,0%	0	7	5
2	30/30	30/30	34.035	8.888	73,9%	896	120	32
3	30/30	30/30	999.825	54.465	94,6%	3.001	3.042	281

Cara membaca: di depth 3, Alpha-Beta hanya memeriksa sekitar 1 dari 18 posisi yang diperiksa Minimax biasa, ±11× lebih cepat, dan tetap memilih skor dan langkah yang sama di 30 dari 30 posisi. Di depth 1 tidak ada penghematan karena belum ada simpul MIN yang bisa dipangkas (AI hanya melihat langkahnya sendiri).

2. Kekuatan AI / win rate (Tabel 2.2.3, baris 2)

Level	Depth	vs pemain acak	vs pemain greedy
Easy	1	99%	89%
Medium	2	99%	100%
Hard	3	100%	79%
Expert	4	100%	92%

Target proposal "Medium ke atas menang minimal 90% melawan pemain acak" **tercapai** (99–100%). Melawan greedy, win rate **tidak naik seiring level**: Medium 100%, tetapi Hard hanya 79%, di bawah Easy (89%). Jadi target "win rate naik seiring level" **tidak tercapai** dengan aturan makan yang sekarang. Penyebabnya belum diteliti dengan uji khusus; yang bisa disebut sebagai dugaan:

- **Kedalaman ganjil dan genap.** Depth 1 dan 3 berhenti setelah langkah AI sendiri, depth 2 dan 4 setelah balasan lawan. Dengan aturan makan, hasilnya bisa berbeda cukup jauh antara ganjil dan genap.
- **Greedy selalu makan kalau bisa**, dan hanya pemain Oranye yang bisa makan. Karena AI bergantian jadi Biru dan Oranye, di separuh game AI hanya bisa menang dengan mengurung.
- Fungsi evaluasi BFS/DFS adalah perkiraan, dan pembatasan 12 kandidat jebakan bisa membuang langkah yang ternyata bagus.
- Dengan 100 game, selisih beberapa poin persen masih bisa karena variasi acak, tetapi selisih Medium–Hard (21 poin) terlalu besar untuk dijelaskan hanya dengan itu.

Kalau ditanya, sampaikan angka ini apa adanya sebagai temuan pengujian, dan jangan menyebut satu penyebab sebagai yang pasti.

3. Waktu respons (Tabel 2.2.3, baris 3)

Level	Langkah diukur	Rata-rata	p95	Maksimum
Easy	957	0,2 ms	0,3 ms	10,1 ms
Medium	826	1,2 ms	4,1 ms	8,0 ms
Hard	795	17,2 ms	50,9 ms	92,2 ms
Expert	842	148,2 ms	443,4 ms	696,1 ms

Target "semua level di bawah 4 detik, Easy–Hard di bawah 1 detik" **tercapai jauh**. Angka ini diukur di Node.js dengan kode yang sama dengan Web Worker di browser; di laptop yang lebih lambat angkanya bisa lebih besar, tetapi masih jauh dari 4 detik. *p95* artinya 95% langkah selesai di bawah angka itu.

4. Hal yang belum dibuktikan oleh tes otomatis

- **Playtest** (±10 siswa, kuesioner skala 1–5) adalah uji manusia; hasilnya harus dicatat sendiri oleh tim. Jangan sebut angka playtest yang tidak benar-benar dilakukan.
- Waktu respons di browser bisa dicek langsung di panel **Statistik AI** saat bermain (kolom Waktu).

tests/run.mjs : baris 16–22

```
16  function check(name, cond, detail = '') {
17    if (cond) passes++;
18    else {
19      failures++;
20      console.log(`  FAIL ${name}${detail ? ' :: ' + detail : ''}`);
21    }
22  }
```

13 Tambahan di Luar Proposal: Makan dan Waktu Giliran

Dua aturan ini **tidak ada di proposal**, tetapi ada di game, jadi bisa ditanya saat demo. Bukan bagian dari empat algoritma AI. Versi sekarang juga sudah tidak punya poin, leaderboard, suara, atau animasi; tampilannya dibuat sederhana.

Makan

Aturan (d) di proposal melarang mendarat di kotak lawan. Di kode, mendarat **tepat** di kotak knight musuh justru **memakan** musuh, seperti di catur: game langsung selesai, tidak ada jebakan, dan yang makan menang. Bersebelahan dengan musuh tidak berarti apa-apa; game tetap jalan.

Tempat	Yang dilakukan
`legalJumps()`, `canMove()`	Kotak musuh dihitung sebagai lompatan legal (cek `cells[n] === EMPTY`, bukan `isOpen`).
`isOpen()`	Tetap melarang jebakan di kotak yang ada knight-nya.
`validateMove()`	Langkah makan sah hanya dengan `trap = -1`.
`winner()`	Dua knight di satu kotak berarti yang sedang jalan baru saja dimakan, jadi lawannya menang.
`Game.handleCell()`	Klik di kotak musuh langsung mengakhiri giliran tanpa fase jebakan.
`ai.js`	`generateMoves()` membuat langkah makan sebagai `pack(j, -1)`; `orderMoves()` memeriksanya paling dulu; `ab()` dan `evaluate()` menilai "dimakan" sebagai kalah dan "bisa makan" sebagai menang di ply berikutnya.

tests/run.mjs : baris 85–96

```
85    // Eating: landing exactly on the enemy's square wins at once, with no trap.
86    const eatBlue = createState({ p1: sq('a1'), p2: sq('c2'), first: 0 });
87    check('blue eats: no trap is valid', validateMove(eatBlue, { jump: sq('c2'), trap: -1 }));
88    check('blue eats: a real trap is rejected', !validateMove(eatBlue, { jump: sq('c2'), trap: sq('a7')
}));
89    applyMove(eatBlue, { jump: sq('c2'), trap: -1 });
90    check('blue eats: blue wins', winner(eatBlue) === 0);
91    check('blue eats: both knights share the square', eatBlue.pos[0] === sq('c2') && eatBlue.pos[1] ===
sq('c2'));
92
93    const eatOrange = createState({ p1: sq('e6'), p2: sq('g7'), first: 1 });
94    check('orange eats: no trap is valid', validateMove(eatOrange, { jump: sq('e6'), trap: -1 }));
95    applyMove(eatOrange, { jump: sq('e6'), trap: -1 });
96    check('orange eats: orange wins', winner(eatOrange) === 1);
```

> **Kenapa hanya Oranye yang bisa makan?** Setiap lompatan knight selalu pindah warna kotak (terang ↔ gelap). a1 dan g7 berwarna sama. Biru selalu jalan duluan, jadi pada giliran Biru kedua knight sudah melompat sama banyak dan berdiri di warna yang sama; lompatan Biru pasti mendarat di warna lain, jadi tidak mungkin kena Oranye. Pada giliran Oranye, Biru sudah melompat sekali lebih banyak, jadi warnanya berbeda dan lompatan Oranye bisa mendarat di kotak Biru. Akibatnya Biru hanya bisa menang dengan mengurung. Mengubah posisi awal hanya menukar sisi mana yang bisa makan.

Waktu giliran

Satu giliran manusia maksimal `TURN_SECONDS` (15 detik, di `config.js`) untuk lompat **dan** jebak sekaligus. Sisa waktu tampil di samping status giliran dan menjadi merah saat tinggal 3 detik.

`js/game.js : baris 165–174`

```
165    tickClock() {
166      const left = this.deadline - performance.now();
167      this.ui.timer(Math.max(0, Math.ceil(left / 1000)));
168      if (this.current.kind === 'human') {
169        if (left <= 0) this.timeUp();
170      } else if (this.role === 'host' && left <= -TURN_GRACE_MS) {
171        this.stopClock();
172        this.netQueue = this.netQueue.then(() => this.playForQuietGuest()).catch((err) =>
console.error(err));
173      }
174    }
```

`js/game.js : baris 178–191`

```
178    timeUp() {
179      this.stopClock();
180      this.ui.timeUp();
181      const s = this.state;
182      if (this.phase === 'jump') {
183        const { jump, trap } = randomMove(s);
184        this.handleCell(jump);
185        if (this.phase === 'trap') this.handleCell(trap);
186      } else if (this.phase === 'trap') {
187        const open = [];
188        for (let sq = 0; sq < SIZE; sq++) if (isOpen(s, sq)) open.push(sq);
189        this.handleCell(open[Math.floor(Math.random() * open.length)]);
190      }
191    }
```

- **Waktu habis sebelum lompat:** `randomMove()` memilih langkah acak yang lengkap (lompatan + jebakan). **Sudah lompat:** lompatanmu tetap, hanya jebakannya yang dipilih acak.
- Langkah acak dimasukkan lewat `handleCell()`, sama seperti klik, jadi aturan dan pengiriman online tidak ditulis dua kali.
- **Batal lompat tidak mengulang waktu:** `deadline` tidak berubah.
- **AI tidak memakai penghitung ini.** AI punya batas 4 detik sendiri (bagian Louis).
- **Online:** perangkat masing-masing pemain bertindak saat waktunya habis. Kalau tamu diam lebih dari `TURN_GRACE_MS` (1,5 detik) setelah batas, host memainkan langkah acak untuknya.
- Bagian 6 di `tests/run.mjs` menguji semua hal di atas dengan batas waktu yang dibuat pendek.

Petunjuk di papan

Kotak yang bisa kamu tuju selalu ditandai titik merah. Kotak yang bisa dituju musuh (di fase jebakan) disembunyikan, dan baru tampil bergaris putus-putus setelah tombol **P** ditekan. Setiap game baru mulai dengan petunjuk musuh tersembunyi.

14 Latihan

Latihan 1: Indeks

Berapa indeks e2 dan f5? Kotak apa yang berindeks 17?

Jawaban: e2: r = 7 − 2 = 5, c = 4 → 39. f5: r = 2, c = 5 → 19. Indeks 17: r = 2, c = 3 → d5.

Latihan 2: Validasi

Knight P1 di d4, P2 di e6, kotak b3 terbakar. Apakah langkah `{ jump: f5, trap: d4 }` sah?

Jawaban: Tidak. Lompatan ke f5 legal, tetapi setelah lompat, d4 terbakar, jadi tidak bisa dijebak. Tes "trap on burned origin rejected" memeriksa kasus ini.

Latihan 4: Makan

Knight P1 di a1, P2 di c2, giliran P1. Apakah `{ jump: c2, trap: a7 }` sah? Bagaimana dengan `{ jump: c2, trap: -1 }`?

Jawaban: Yang pertama tidak sah: lompat ke c2 memakan P2, jadi tidak boleh ada jebakan. Yang kedua sah, dan setelahnya `winner()` mengembalikan 0 (P1 menang) karena kedua knight ada di c2.

Latihan 3: Flood fill

Pada contoh papan terbelah, area P1 = {b3, c2} dan P2 = {e6, f5, g3, g5}. Kenapa P1 tidak bisa mencapai f5 walaupun f5 terbuka?

Jawaban: Semua jalur dari b3/c2 ke f5 melewati kotak terbakar. DFS hanya menyebar ke tetangga yang terbuka, jadi f5 tidak pernah dicapai dari a1.

15 Pertanyaan Presentasi yang Mungkin Muncul

T: Kenapa papan disimpan sebagai array satu dimensi, bukan 7 × 7?

J: Lebih cepat dan sederhana: satu angka per kotak, mudah disalin (`Int8Array`), dan tetangga knight sudah dihitung sebagai daftar indeks.

T: Kenapa DFS untuk flood fill, bukan BFS?

J: Untuk flood fill hanya perlu tahu kotak mana yang terhubung, bukan jaraknya. Keduanya benar; DFS dengan stack sedikit lebih sederhana. Proposal memakai keduanya untuk tujuan berbeda: BFS untuk jarak/wilayah, DFS untuk area/split.

T: Bagaimana game mencegah langkah curang?

J: Semua langkah melewati `legalJumps` / `isOpen`; langkah dari AI dan dari pemain online diperiksa dengan `validateMove`, dan host menolak langkah tamu yang tidak sah. Aturan hanya ditulis sekali di `rules.js`.

T: Proposal melarang mendarat di kotak lawan. Kenapa di game boleh?

J: Itu perubahan setelah proposal: mendarat di kotak lawan sekarang memakan lawan dan langsung menang. Karena warna kotak, hanya Oranye (yang jalan kedua) yang bisa makan. Jebakan tetap tidak boleh di kotak knight.

T: Apa arti "papan terbelah" dan kenapa skornya × 100?

J: Kedua knight tidak lagi bisa mencapai kotak yang sama, jadi hasil game praktis ditentukan oleh luas area. × 100 memastikan situasi ini selalu dinilai lebih penting daripada skor wilayah (maks. ±53).

T: Apa yang terjadi kalau tidak ada kotak untuk dijebak?

J: Jebakan dilewati dan dicatat sebagai −1. `validateMove` hanya menerima −1 dalam kasus ini.

T: Apa hasil playtest kalian?

J: Laporkan angka nyata dari kuesioner. Kalau playtest belum selesai, katakan itu dengan jujur dan jelaskan rencananya.

16 Checklist Sebelum Presentasi

- ☐ Bisa mengubah koordinat ke indeks dan sebaliknya tanpa kalkulator.
- ☐ Bisa menunjuk fungsi untuk setiap aturan (a)–(e) di `rules.js`, plus aturan makan.
- ☐ Bisa menjelaskan flood fill dengan contoh papan terbelah.
- ☐ Bisa menjelaskan urutan DFS → BFS di `evaluate()` dan alasan bobot 100.
- ☐ Data playtest sudah direkap (jumlah peserta, rata-rata, bug).
- ☐ Bisa menjelaskan alur satu giliran (Bab 4) tanpa melihat catatan.